da24c006

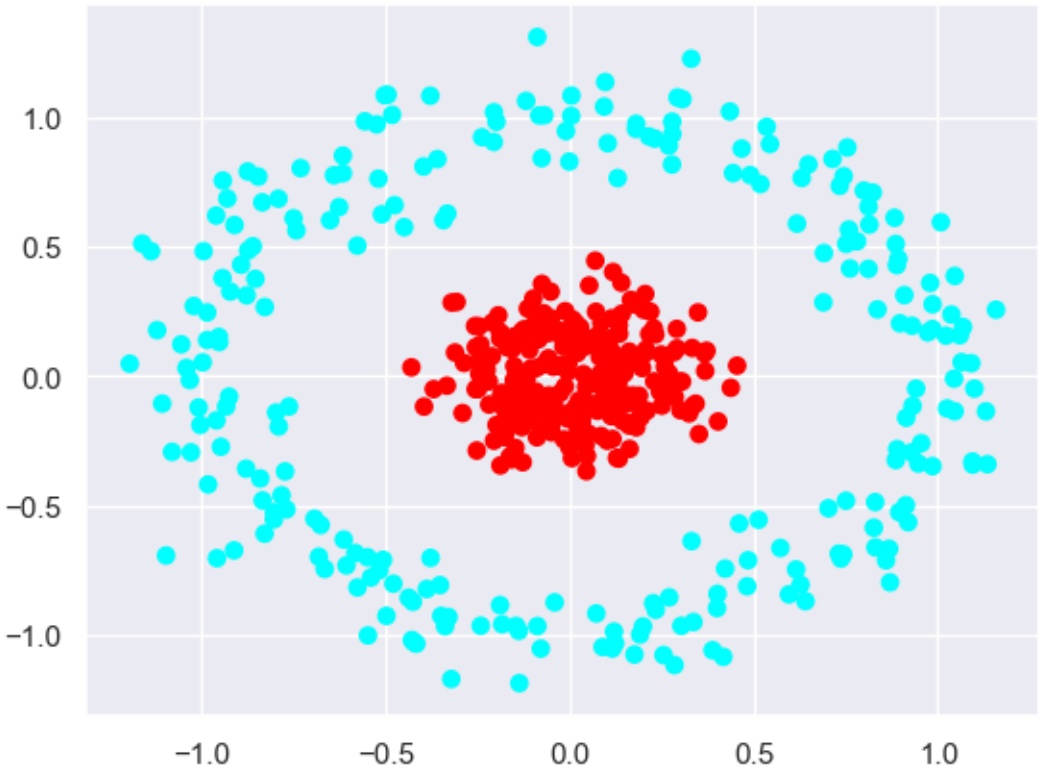
October 13, 2024

[7]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.colors as colors

from sklearn.datasets import make_circles
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.base import BaseEstimator, ClassifierMixin, clone
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import LinearSVC
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

import seaborn as sns
sns.set_theme()
```

[8]:
```
X, y=make_circles(n_samples=500, noise=0.1, random_state=42, factor=0.2)
y=2*y-1
X_train, X_test, y_train, y_test=train_test_split(X, y, random_state=42)
plt.scatter(X[:, 0], X[:, 1], c=y+2, cmap=colors.ListedColormap(['cyan',␣
 ↪'red']));
```



[3]:
```
def plot_decision_boundary(model, X, y, ax=None, alpha=1, plot_points=True):
    '''
    This function plots the decision boundary of a classifier.

    #### Parameters

    `model` : This is the classifier for which the decision boundary has to be␣
↪plotted.

    `X` : The  data matrix used to plot the decision boundary. Each row is a␣
↪data point, each column is a feature.

    `y` : The label vector for the data matrix. This will be used to color the␣
↪boundary.

    `ax` : matplotlib.pyplot.Axes() instance. May be provided to draw on a␣
↪specific axes object.

    `alpha` : This controls the transparency of the decision boundary.
```

```
    `plot_points`: If True, then the individual data points will also be␣
↪plotted according to their true label.

    #### Returns
    If `ax` is None then a new axes object is returned which contains the plot.␣
↪Otherwise the passed refrence is returned.
    '''

    if ax is None:
        ax=plt.axes()
    points_x=np.linspace(X[:, 0].min(), X[:, 0].max(), 100)
    points_y=np.linspace(X[:, 1].min(), X[:, 1].max(), 100)
    mesh=np.meshgrid(points_x, points_y)
    df=pd.DataFrame(np.column_stack((mesh[0].flatten(), mesh[1].flatten())))

    predictions=model.predict(df)
    ax.scatter(df[0], df[1], s=5, marker='.', c=predictions+2,
               cmap=colors.ListedColormap(['blue', 'yellow']), alpha=alpha)
    if plot_points:
        ax.scatter(X[:, 0], X[:, 1], c=y+2, cmap=colors.ListedColormap(['cyan',␣
↪'orange']), edgecolors='k')

    return ax
```

1 Task 1

[216]:
```
class AdaBoost(BaseEstimator, ClassifierMixin):
    def __init__(self, estimator, learning_rate=0.5, n_estimators=10,␣
↪random_state=0):
        self.estimator=estimator # the learner that will be used
        self.learning_rate=learning_rate # the learning rate that will be used
        self.n_estimators=n_estimators # the number of max estimators to add in␣
↪the ensemble
        self.actual_iterations=0 # actual iterations took to fit the model
        self.estimators=[]  # the list of trained estimators
        self.weights=[] # alphas, weight of each estimator
        self.random_state=random_state
        self._is_fitted=False

    def calc_error(self, y_true, y_pred, distribution):
        '''
        This function returns the error of the current interation based on the␣
↪true labels, predicted labels and
        the current distribution.

        #### Parameters
```

```
        `y_true` : The true labels of classification.

        `y_pred` : The predicted labels of classification in the current␣
↪iteration.

        `distribution` : The current distribution of data points.
        '''

        return ((y_true != y_pred)*distribution).sum()

    def calc_weight(self, error):
        '''
        This function calculates the weight of the current iteration based on␣
↪the error returned by `calc_error`.
        If error is 0, then 10 (a large value) is returned. If error is at␣
↪least half, then 0.01 (a smaal value is returned).
        Otherwise the value is calculted using the usual formula.
        '''

        if error==0:
            return 10
        elif error>=0.5:
            return 0.01
        else:
            return self.learning_rate * np.log2((1-error)/error)

    def updated_distribution(self, y_true, y_pred, distribution, new_alpha):
        '''
        This function calculates the updated distribution for the next␣
↪iteration using the true labels,
        predicted labels, distribution and the weight (alpha) of the current␣
↪iteration.

        #### Parameters

        `y_true` : The true labels
        `y_pred` : The predicted labels
        `distribution` : The current distribution of the data points
        `new_alpha` : The weight (alpha) of the current distribution
        '''
        un_normalised=np.exp((-new_alpha)*y_true*y_pred)*distribution
        return un_normalised/un_normalised.sum()


    def fit(self, X, y):
        # initialization steps
```

```
        n=X.shape[0]
        self.estimators=[]
        self.weights=[]
        self.actual_iterations=0
        distribution=np.array([1/n]*n) # initially the distribution is uniform

        for i in range(self.n_estimators):
            new_estimator=clone(self.estimator)
            new_estimator.fit(X, y, sample_weight=distribution) # fit according␣
↪to the distribution
            predictions=new_estimator.predict(X)
            error=self.calc_error(y, predictions, distribution) # calculate the␣
↪error
            new_alpha=self.calc_weight(error) # calculate the weight

            self.estimators.append(new_estimator)
            self.weights.append(new_alpha)
            self.actual_iterations+=1

            if error==0:
                break

            distribution=self.updated_distribution(y, predictions,␣
↪distribution, new_alpha) # update the distribution

        self._is_fitted=True
        return self

    def predict(self, X):
        ensemble_predictions=np.zeros(X.shape[0])
        for i in range(self.actual_iterations):
            ensemble_predictions+=self.weights[i]*self.estimators[i].predict(X)

        positive=(ensemble_predictions>=0)
        ensemble_predictions[positive]=1
        ensemble_predictions[~positive]=-1
        return ensemble_predictions

    def plot_decision_boundary_pair(self, X, y, alpha=1):
        '''
        This function plots the decision boundary for each intermediate␣
↪estimator and the whole ensemble.
        '''
        fig, axes=plt.subplots(ncols=2)
        for i in range(self.actual_iterations):
            points=False
            if i==self.actual_iterations-1:
```

```
                points=True
            plot_decision_boundary(self.estimators[i], X, y, ax=axes[0],
                                   alpha=alpha, plot_points=points)
        plot_decision_boundary(self, X, y, ax=axes[1], alpha=0.4)
        axes[0].set_xlabel('$X_1$')
        axes[0].set_ylabel('$X_2$')
        axes[1].set_xlabel('$X_1$')
        axes[1].set_ylabel('$X_2$')
        return fig, axes


    def __sklearn_is_fitted__(self):
        return self._is_fitted
```

[8]:
```
dt=DecisionTreeClassifier(max_depth=1)
dt.fit(X_train, y_train)
```

[8]:
```
DecisionTreeClassifier(max_depth=1)
```

[78]:
```
clsf=AdaBoost(dt, n_estimators=1000)
clsf.fit(X_train, y_train)
```

[78]:
```
AdaBoost(estimator=DecisionTreeClassifier(max_depth=1), n_estimators=1000)
```

[79]:
```
clsf.actual_iterations
```

[79]:
```
1000
```

[80]:
```
predictions=clsf.predict(X_train)
(predictions==y_train).mean()
```

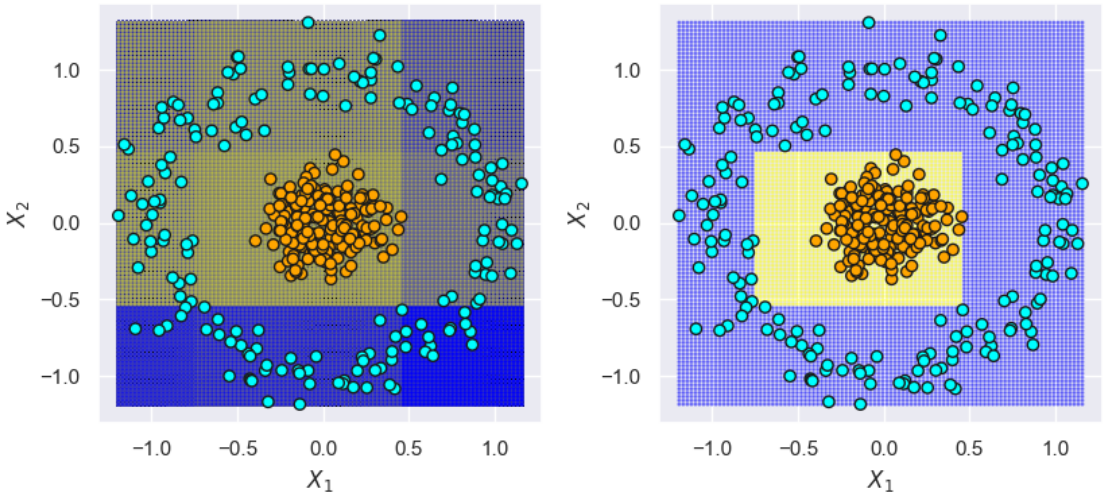
[80]:
```
np.float64(1.0)
```

[81]:
```
predictions=clsf.predict(X_test)
(predictions==y_test).mean()
```

[81]:
```
np.float64(1.0)
```

The Decision Boundary We can see that the model gives perfect accuracy and the combined discion boundary is composed of straight lines. This is because the underlying model is a Decision Tree which has a decisoin boundary of straight lines.

[82]:
```
fig, axes=clsf.plot_decision_boundary_pair(X_train, y_train, alpha=0.4)
fig.set_size_inches(8.5, 4)
plt.tight_layout()
```



2 Task 2

[278]:
```
def model_report(model):
    '''
    This function prints the model accuracy on the Training and Test Set
    '''
    print('Train Performance:')
    predictions=model.predict(X_train)
    score=np.mean((predictions==y_train))*100
    print('\tAccuracy Score: %.3f'%score)

    print('Test Performance:')
    predictions=model.predict(X_test)
    score=np.mean((predictions==y_test))*100
    print('\tAccuracy Score: %.3f'%score)
```

The following sections tunes the Boosting model with different underlying weak learners. In each section the best parameters are found using Grid Search and the Decision Boundary is visualised.

2.0.1 Decision Stump

[275]:
```
gscv=GridSearchCV(estimator=AdaBoost(estimator=DecisionTreeClassifier(max_depth=1)),
                  param_grid={'learning_rate': [0.4, 0.5, 0.6],
                              'n_estimators': [20, 50, 100, 200, 250]},
                  cv=5, refit=False, n_jobs=-1)
gscv.fit(X_train, y_train)
```

[275]:
```
GridSearchCV(cv=5,
             estimator=AdaBoost(estimator=DecisionTreeClassifier(max_depth=1)),
```

```
                n_jobs=-1,
                param_grid={'learning_rate': [0.4, 0.5, 0.6],
                            'n_estimators': [20, 50, 100, 200, 250]},
                refit=False)
```

[276]:
```
print('Best Params:' + repr(gscv.best_params_))
print('Best Score: %.3f'%gscv.best_score_)
```

```
Best Params:{'learning_rate': 0.4, 'n_estimators': 20}
Best Score: 0.989
```

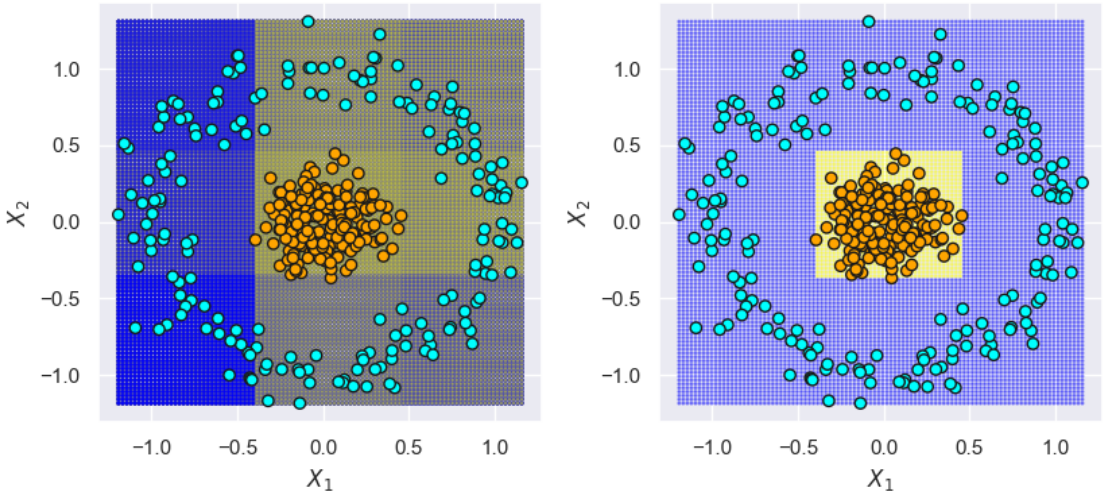
[277]:
```
clsf=AdaBoost(estimator=DecisionTreeClassifier(max_depth=1), **gscv.
  ↪best_params_)
clsf.fit(X_train, y_train)
```

[277]:
```
AdaBoost(estimator=DecisionTreeClassifier(max_depth=1), learning_rate=0.4,
         n_estimators=20)
```

[279]:
```
model_report(clsf)
```

```
Train Performance:
        Accuracy Score: 100.000
Test Performance:
        Accuracy Score: 99.200
```

[281]:
```
fig, axes=clsf.plot_decision_boundary_pair(X_train, y_train, alpha=0.4)
fig.set_size_inches(8.5, 4)
plt.tight_layout()
```



2.0.2 Descision Tree with max_depth=3

[282]:
```
gscv=GridSearchCV(estimator=AdaBoost(estimator=DecisionTreeClassifier(max_depth=3)),
                  param_grid={'learning_rate': [0.4, 0.5, 0.6],
                              'n_estimators': [20, 50, 100, 200, 250]},
                  cv=5, refit=False, n_jobs=-1)
gscv.fit(X_train, y_train)
```

[282]:
```
GridSearchCV(cv=5,
             estimator=AdaBoost(estimator=DecisionTreeClassifier(max_depth=3)),
             n_jobs=-1,
             param_grid={'learning_rate': [0.4, 0.5, 0.6],
                         'n_estimators': [20, 50, 100, 200, 250]},
             refit=False)
```

[283]:
```
print('Best Params:' + repr(gscv.best_params_))
print('Best Score: %.3f'%gscv.best_score_)
```

```
Best Params:{'learning_rate': 0.4, 'n_estimators': 200}
Best Score: 0.995
```

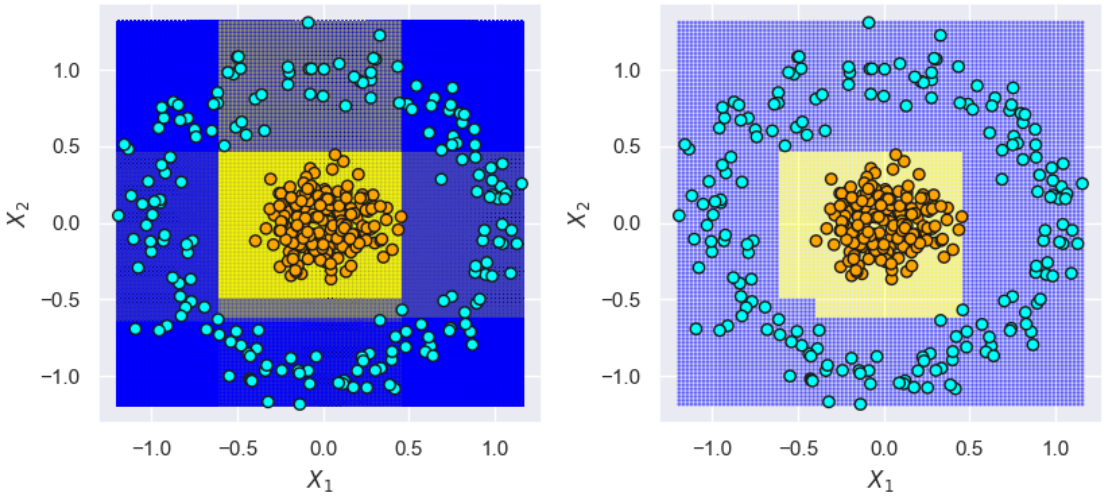
[284]:
```
clsf=AdaBoost(estimator=DecisionTreeClassifier(max_depth=3), **gscv.
 ↪best_params_)
clsf.fit(X_train, y_train)
```

[284]:
```
AdaBoost(estimator=DecisionTreeClassifier(max_depth=3), learning_rate=0.4,
         n_estimators=200)
```

[285]:
```
model_report(clsf)
```

```
Train Performance:
        Accuracy Score: 100.000
Test Performance:
        Accuracy Score: 100.000
```

[286]:
```
fig, axes=clsf.plot_decision_boundary_pair(X_train, y_train, alpha=0.4)
fig.set_size_inches(8.5, 4)
plt.tight_layout()
```



2.0.3 Linear SVM

[287]:
```
gscv=GridSearchCV(estimator=AdaBoost(estimator=LinearSVC()),
                  param_grid={'learning_rate': [0.4, 0.5, 0.6],
                              'n_estimators': [20, 50, 100, 200, 250]},
                  cv=5, refit=False, n_jobs=-1)
gscv.fit(X_train, y_train)
```

[287]:
```
GridSearchCV(cv=5, estimator=AdaBoost(estimator=LinearSVC()), n_jobs=-1,
             param_grid={'learning_rate': [0.4, 0.5, 0.6],
                         'n_estimators': [20, 50, 100, 200, 250]},
             refit=False)
```

[288]:
```
print('Best Params:' + repr(gscv.best_params_))
print('Best Score: %.3f'%gscv.best_score_)
```

```
Best Params:{'learning_rate': 0.4, 'n_estimators': 250}
Best Score: 0.949
```

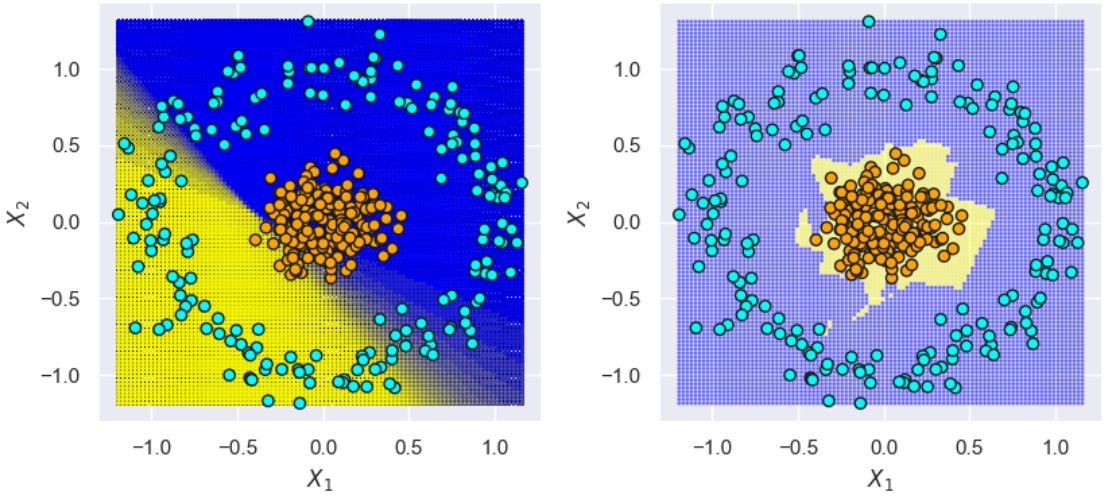
[293]:
```
clsf=AdaBoost(estimator=LinearSVC(), **gscv.best_params_)
clsf.fit(X_train, y_train)
```

[293]:
```
AdaBoost(estimator=LinearSVC(), learning_rate=0.4, n_estimators=250)
```

[294]:
```
model_report(clsf)
```

```
Train Performance:
        Accuracy Score: 100.000
Test Performance:
        Accuracy Score: 99.200
```

[295]:
```
fig, axes=clsf.plot_decision_boundary_pair(X_train, y_train, alpha=0.1)
fig.set_size_inches(8.5, 4)
plt.tight_layout()
```



2.0.4 Logistic Regression

[296]:
```
gscv=GridSearchCV(estimator=AdaBoost(estimator=LogisticRegression()),
                  param_grid={'learning_rate': [0.4, 0.5, 0.6],
                              'n_estimators': [20, 50, 100, 200, 250]},
                  cv=5, refit=False, n_jobs=-1)
gscv.fit(X_train, y_train)
```

[296]:
```
GridSearchCV(cv=5, estimator=AdaBoost(estimator=LogisticRegression()),
             n_jobs=-1,
             param_grid={'learning_rate': [0.4, 0.5, 0.6],
                         'n_estimators': [20, 50, 100, 200, 250]},
             refit=False)
```

[297]:
```
print('Best Params:' + repr(gscv.best_params_))
print('Best Score: %.3f'%gscv.best_score_)
```

```
Best Params:{'learning_rate': 0.4, 'n_estimators': 250}
Best Score: 1.000
```

[298]:
```
clsf=AdaBoost(estimator=LogisticRegression(), **gscv.best_params_)
clsf.fit(X_train, y_train)
```

[298]:
```
AdaBoost(estimator=LogisticRegression(), learning_rate=0.4, n_estimators=250)
```

[299]:
```
model_report(clsf)
```

```
Train Performance:
        Accuracy Score: 100.000
Test Performance:
        Accuracy Score: 100.000
```

[301]:
```
fig, axes=clsf.plot_decision_boundary_pair(X_train, y_train, alpha=0.1)
fig.set_size_inches(8.5, 4)
plt.tight_layout()
```